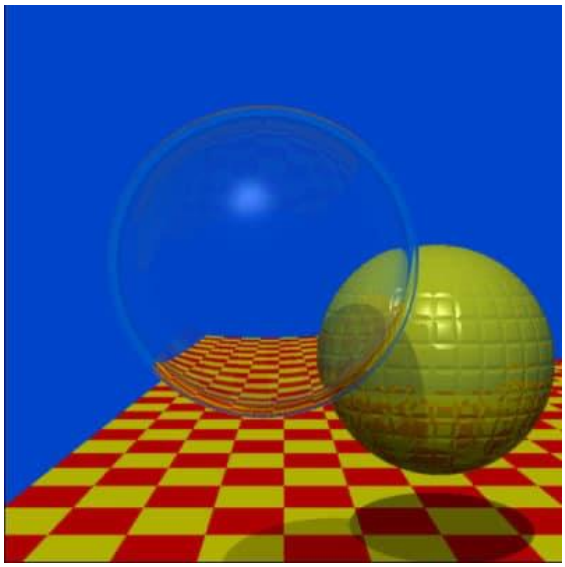
CSCE 4813 – Programming Project 6
Due Date – Friday April 24th, 2026 at 11:59pm

1. Problem Statement:

Ray tracing is capable of creating some amazingly realistic images of objects because it can simulate reflection, refraction, and many illumination effects. This realism comes at a cost, because in order to calculate the color of each pixel in the output image we have to cast one or more rays into the scene and calculate the intersection point with the object closest to the camera, and then recursively cast reflection and refraction rays to determine what is visible at each point.



Still from Whitted's animation: https://youtu.be/0KrCh5qD9Ho

We have three sample ray tracing programs in the src directory. The "ray_trace" program displays a single sphere, "ray_trace2" displays N stationary spheres with random colors in random locations, and "ray_trace3" displays N moving spheres that bounce off the boundary of the image.

In this project, you will be extending one of these ray tracing programs to learn more about ray tracing internals as you add several new features. In particular, you are required to add **three** of the following four features to your program.

Feature 1 – Reproduce Turner Whitted's Sphere Animation

If you watch the vintage demo of Turner Whitted's original ray tracing animation you will see a solid yellow ball moving around a transparent stationary ball over a checkerboard background. To reproduce this animation, you need to use the "idle" or "timer" callback in OpenGL to update the position of the solid ball and generate and display the ray traced image.

Try to get the sizes of the spheres and the motion as close to the demo as possible. Displaying the checkerboard pattern on the floor is a little tricky. When you intersect the ray with the floor plane, you can look at the (x,y,z) coordinates of the intersection point to calculate the color to display (red/yellow/empty). You are not required to implement sphere transparency for this feature.

Feature 2 – Display different objects

There are not many realistic scenes that consist of only spheres, so it would be nice to extend the ray_trace program to display some different kinds of geometric objects. If you look at the lecture notes and Wikipedia, you will find the necessary formulas to intersect rays with planar polygons, cubes, cylinders, and cones.

Pick **one** of these geometric objects and add the necessary class and methods to intersect rays with this object. Then extend the ray_trace program to display two or more of these objects in your program.

Feature 3 – Cast reflection rays (mirrors)

The current ray tracing programs cast two rays per output pixel, one ray to determine which object is closest to the camera, and one ray from that intersection point towards the light source to see if the point is in shadow. To make more realistic images, you can add reflection rays.

To do this, you will have to calculate the ideal reflection direction, and cast a new ray in this direction and calculate the color of the object that is intersected (if any). Then you must add Ks times the observed color to the Phong shaded object color and store this in the output pixel. If you are feeling ambitious, you could cast a second reflection ray and do two levels of reflection.

Feature 4 – Cast transmission rays (glass)

The current ray tracing programs cast two rays per output pixel, one ray to determine which object is closest to the camera, and one ray from that intersection point towards the light source to see if the point is in shadow. To make more realistic images, you can add transmission rays.

To do this, you will have to make assign an index of refraction to each sphere in the scene, and calculate the transmission direction using Snell's formulas. Then cast a new ray in this direction and calculate the color of the object that is intersected (if any). Finally, you must add Kt times the observed color to the Phong shaded object color and store this in the output pixel. If you are feeling ambitious, you could cast a second reflection ray and do two levels of refraction.

2. Design:

Your first design task is to do some research on the math required to implement the two features you have selected. The lecture notes and the Wikipedia page on ray tracing are good places to start. Once you have the math worked out, you should look over the code in ray_classes to see what classes/methods you can reuse, and what new classes/methods need to be added.

Your next design task is to decide what additional data structures will be needed to keep track of the moving spheres, new objects, and your light sources, and come up with algorithms to initialize and use these data structures. With OpenGL applications, you may need to make these global variables.

Finally, you need to come up with a plan for incrementally adding the features you have chosen. You may want to print or display intermediate information as you go. For example, you may want to work on a small portion of the output image, and print out the object ids for the spheres you intersect, and the ideal reflection direction before you actually trace the reflection arrays. When implementing multiple light sources, you could create an output image with the color from each light source separately before you try to combine them.

3. Implementation:

This semester we will be using C++ and OpenGL to implement all of our programming projects. The instructions for creating and compiling OpenGL programs are posted in README file the "Source Code" page of the class website.

You are encouraged to look at sample OpenGL programs to see how the "main" function and the "display" function are normally implemented. As always, you should break the code into appropriate functions, and then add code incrementally writing comments, adding code, compiling, debugging, a little bit at a time.

Remember to use good programming style when creating your program. Choose good names for variables and constants, use proper indenting for loops and conditionals, and include clear comments in your code. Also, be sure to save backup copies of your program somewhere safe. Otherwise, you may end up retyping your whole program if something goes wrong.

4. Testing:

Test your program with different random number generator seeds until you get some images that look fun/interesting. Take a screen shot of these images to include in your project report. You may also want to show some bad/ugly images that illustrate what happens if there is a problem somewhere. You can discuss how you corrected these problems in your project report.

5. Documentation:

When you have completed your C++ program, write a short report using the project report template describing what the objectives were, what you did, and the status of the program. Be sure to include several output images. Finally, describe any known problems and/or your ideas on how to improve your program. Save this report to be submitted electronically via Blackboard.

6. Project Submission:

In this class, we will be using electronic project submission to make sure that all students hand their programming projects and labs on time, and to perform automatic plagiarism analysis of all programs that are submitted. When you have completed the tasks above go to Blackboard to upload your documentation (a single docx or pdf file), and all of your C++ program files. Do NOT upload an executable version of your program.

The dates on your electronic submission will be used to verify that you met the due date above. All late projects will receive reduced credit:

- 25% off if less than 1 day late,
- 50% off if less than 2 days late,
- 75% off if less than 3 days late,
- no credit if more than 3 days late.

You will receive partial credit for all programs that compile even if they do not meet all program requirements, so handing projects in on time is highly recommended.

7. Academic Honesty Statement:

Students are expected to submit their own work on all programming projects, unless group projects have been explicitly assigned. Students are NOT allowed to distribute code to each other or copy code from another individual or website. Students ARE allowed to use any materials on the class website, or in the textbook, or ask the instructor and/or GTAs for assistance.

<u>For this project you ARE allowed to use generative AI to learn about OpenGL and to help you write the specified code above. You are required to keep track of all of your AI prompts in a separate document to submit with your project report. Note: You can ask the AI to keep track of all of your prompts for you and then ask it to produce this document.</u>

This course will be using highly effective program comparison software to calculate the similarity of all programs to each other, and to homework assignments from previous semesters. Please do not be tempted to plagiarize from another student.

Violations of the policies above will be reported to the Provost's office and may result in a ZERO on the programming project, an F in the class, or suspension from the university, depending on the severity of the violation and any history of prior violations.